

ITERATIVE ALGORITHMS FOR THE HERMITIAN POSITIVE DEFINITE SOLUTION OF THE RICCATI MATRIX DIFFERENTIAL EQUATION ARISING IN DIFFERENTIAL GAME MODELS*

JIANZHOU LIU[†], ZE ZHANG[†], AND YUNLAN WEI[‡]

Abstract. The Riccati matrix differential equation (RDE) holds immense importance in modern control theory and practical engineering applications. This paper first derives a novel iterative algorithm for solving the RDE based on a matrix series. Then, by exploiting the properties of matrix series and matrix powers, we propose an accelerated algorithm that substantially reduces the computational complexity compared to the original algorithm. Furthermore, we present a detailed analysis of the convergence, error, and computational complexity of these proposed algorithms. Finally, the effectiveness and advantages of these approaches are validated through a series of numerical experiments.

Key words. Riccati matrix differential equation, iterative algorithm, matrix series, error analysis, computational complexity.

AMS subject classifications. 15A24; 15A42; 65F45; 93C05

1. Introduction.

1.1. Research Background and Literature Review. The Riccati matrix differential equation (RDE) plays a pivotal role in modern control theory and numerous scientific applications. It is intimately connected to finite-time optimal control problems, model order reduction, optimal filtering, H_∞ control of linear time-varying systems, and differential game problems [1, 3, 23, 24, 36]. For example, a paradigmatic example arises in differential game theory: the pursuit-evasion game between two spacecraft. In this scenario, the pursuer aims to minimize the relative distance to the evader, while the evader seeks to maximize it, forming a zero-sum dynamic. This problem is particularly significant in aerospace engineering, with applications in space defense, satellite interception, and orbital rendezvous [16], and deriving optimal control strategies for both agents is essential for achieving mission objectives and ensuring operational efficiency, which involves solving an RDE.

Given the wide applicability of the Riccati equation across various domains, numerous researchers have made significant contributions to developing solution algorithms. We begin with a brief review of the continuous algebraic Riccati equation (CARE) and its coupled variants, such as the coupled continuous algebraic Riccati equation (CCARE). The problem of solving the CARE can be transformed into finding a stable invariant subspace of a Hamiltonian matrix, as introduced in [26]. A Newton-like method with exact line search for solving the CARE was presented in [5]. Subsequently, a structure-preserving doubling algorithm was proposed in [13]. An extended block Arnoldi algorithm was introduced for approximating solutions to large-scale CARE in [19]. Moreover, a structure-preserving doubling algorithm was adapted for the CCARE in [37], with two

*Submitted to the editors DATE. (Jianzhou Liu and Ze Zhang contributed equally to this work.) (Jianzhou Liu and Ze Zhang are co-first authors.) (Corresponding author: Jianzhou Liu.)

Funding: This research was partially supported by the National Key R&D Program of China (Grant No. 2023YFB3001604), the National Natural Science Foundation of China (Grant No. 11971413), the Natural Science Foundation of Shandong Province (Grant No. ZR2022QA057), the Project for Scientific Research Fund of Hunan Provincial Education Department (Grant No. 24C0064), the 111 Project (Grant No. D23017).

[†]Key Laboratory of Intelligent Computing and Information Processing of Ministry of Education & School of Mathematics and Computational Science, Xiangtan University, Xiangtan, Hunan, People's Republic of China (li-ujz@xtu.edu.cn, zezhang@smail.xtu.edu.cn.edu).

[‡]School of Mathematics and Statistics, Linyi University, Linyi, Shandong, People's Republic of China (wy119910110@163.com).

improved variants also proposed. A Newton iteration method for the CCARE was presented in [28], and more recently, a class of fixed-point iterations was developed in [38].

Beyond its inherent nonlinearity, solving the RDE involves numerically integrating high dimensional matrices, leading to significant computational complexity. Consequently, developing efficient methods for solving the RDE has become a key research focus. Existing numerical methods for the RDE include the backward differentiation formula (BDF) method [15, 7, 30], the Rosenbrock (ROS) method [8, 9], splitting methods [18, 33, 29, 31, 34, 32, 12], and the peer method [6]. Additional approaches, such as Krylov subspace methods and the Galerkin approach, have also been explored [4, 25]. Recently, large-scale matrix differential equations have been tackled using integral formulas or numerical ODE solvers, as discussed in [10, 11].

1.2. Research Motivation and Potential Issues. As we all know, the classical methods for solving the matrix differential equations whose solutions are full rank can be divided into two categories: (1) unroll matrices into vectors and integrate the resulting system (e.g., MATLAB ODE solvers such as ode45, ode15s, ode23tb, etc.), and (2) matrix-based methods for ODEs, such as the BDF and ROS methods. The first approach faces a significant challenge in that unrolling the matrices into vectors leads to a system of n^2 differential equations, which must be solved using numerical integration techniques. This becomes computationally impractical for large-scale problems. The second approach, while more efficient in terms of matrix representation, involves solving one or more Sylvester matrix equations at each time step, which makes it computationally expensive, and difficult to implement. Therefore, the primary focus of this article is to develop iterative algorithms for solving the RDE, accompanied by a thorough theoretical analysis, while ensuring that the Hermitian and positive semidefinite properties of the solution are preserved, and are more suitable for solving control problems.

1.3. Problem Description. Let us consider the specific differential game model of spacecraft pursuit and evasion mentioned above, where the dynamics are expressed in linear state-space form [35]:

$$\dot{x}(t) = Ax(t) + Bu_p(t) - Bu_e(t),$$

where $x(t)$ is the state vector, $A \in \mathbb{R}^{m \times m}$ is the system matrix, and $B \in \mathbb{R}^{m \times p}$ is the control input matrix. The vectors $u_p(t)$ and $u_e(t)$ represent the control inputs of the pursuer and evader, respectively. The pursuer's objective is to minimize the cost function:

$$J = \frac{1}{2}x^T(t_f)P_fx(t_f) + \frac{1}{2} \int_0^{t_f} (u_p^T(t)R_pu_p(t) - u_e^T(t)R_eu_e(t)) dt,$$

where t_f is the terminal time, $P_f \in \mathbb{R}^{m \times m}$ is the terminal weighting matrix, and $R_p \in \mathbb{R}^{p \times p}$ and $R_e \in \mathbb{R}^{p \times p}$ are the control weighting matrices for the pursuer and evader, respectively. Conversely, the evader aims to maximize this cost function.

The optimal control strategies for both agents are derived from the solution to the following RDE:

$$-\dot{P}(t) = A^TP(t) + P(t)A - P(t)B(R_p^{-1} - R_e^{-1})B^TP(t),$$

with the terminal condition $P(t_f) = P_f$. The solution $P(t)$ facilitates the construction of state feedback control laws:

$$u_p(t) = -R_p^{-1}B^TP(t)x(t), \quad u_e(t) = -R_e^{-1}B^TP(t)x(t).$$

These control laws enable both agents to dynamically adjust their strategies based on the real-time state, optimizing performance relative to the defined cost function.

Hence, without loss of generality, we focus on studying the following form of the RDE in this paper:

$$(1.1) \quad \begin{cases} -\dot{P}(t) = A^*P(t) + P(t)A - P(t)BR^{-1}B^*P(t), & t \in [t_0, t_f], \\ P(t_f) = P_f = P_f^*, \end{cases}$$

where $A \in \mathbb{C}^{m \times m}$, $B \in \mathbb{C}^{m \times p}$, $R \in \mathbb{C}^{p \times p}$ is Hermitian positive definite, and $P_f \in \mathbb{C}^{m \times m}$ is Hermitian. $P(t) \in \mathbb{C}^{m \times m}$ is the Hermitian solution to the RDE (1.1).

1.4. Main Result and Contributions.

1.4.1. A novel expression for the solution of the RDE based on a convergent matrix series. In this paper, it is proven that the solution to the RDE can be represented by a convergent matrix series (Theorem 3.1). Based on this theorem, an algorithm is proposed for solving the Hermitian positive definite solution of the RDE. This theorem ensures the convergence of these proposed algorithms in this paper and provides a foundation for the subsequent error analysis and computational complexity analysis.

1.4.2. Two novel iterative algorithms for solving the RDE. This paper presents innovative iterative algorithms designed to solve the RDE (1.1), a critical tool in control theory and differential game applications such as spacecraft pursuit-evasion. These algorithms utilize a matrix series that iteratively approximates the solution, offering a practical method to tackle complex, high-dimensional systems. When compared to established methods like the BDF and ROS techniques, as well as MATLAB's ode solvers, the proposed algorithms exhibit clear advantages in both speed and accuracy. Numerical experiments reveal that they achieve significant improvement in error precision and faster computation times across various test cases. This superior performance underscores the practical value of the new methods.

1.4.3. A complete theoretical analysis of the derived algorithm. A key strength of this work lies in its thorough analysis of the algorithms' performance, including their convergence behavior, error analysis, and computational complexity discussion. We demonstrate that the iterative process reliably converges to the exact solution, while also providing detailed estimates of the errors introduced during computation. Additionally, we analyze the computational effort required, showing how our approach optimizes resource use, making it suitable for practical problems where efficiency is paramount.

1.5. Organization. The structure of this paper is organized as follows: In Section 2, we introduce some foundational lemmas. Section 3 is dedicated to deriving a novel iterative algorithm based on a matrix series for solving the RDE (1.1). In addition, we give the convergence analysis for the derived algorithm. In Section 4, we provide the error analysis for our method, then based on the given error analysis, by using the properties of matrix series and matrix powers, an accelerated algorithm is obtained, accompanied by an evaluation of the computational complexity for both algorithms. Subsequently, Section 5 presents a series of numerical experiments that illustrate the effectiveness and superiority of our proposed algorithms. Finally, as is standard, Section 6 concludes the paper with a comprehensive summary.

Notations: Let $\mathbb{C}^{m \times m}$ denote the set of $m \times m$ complex matrices. A matrix M is stable means its eigenvalues all have negative real parts. Given a matrix $M \in \mathbb{C}^{m \times m}$, we assume that

the real parts of its eigenvalues are arranged such that $(\operatorname{Re}\lambda_1(M) \geq \operatorname{Re}\lambda_2(M) \geq \dots \geq \operatorname{Re}\lambda_m(M))$ and its singular values are arranged in non-increasing order $(\sigma_1(M) \geq \sigma_2(M) \geq \dots \geq \sigma_m(M))$. M^* , M^{-1} , $\rho(M)$, $\overline{\lambda}_j(M)$, $\operatorname{Im}\lambda_j(M)$, and $|\lambda_j(M)|$ denote the conjugate transpose, inverse, spectral radius, conjugate of j -th eigenvalue, imaginary part, modulus of the j -th eigenvalue for a matrix M , respectively. The symbol $\|M\|$ means the matrix norm of a matrix M (in particular, $\|M\|_2$ and $\|M\|_F$ represent the spectral and Frobenius norms, respectively). 0_m and I_m represent the null matrix and identity matrix with order m , respectively. $\mathbb{H}_m$ represents the set of $m \times m$ Hermitian matrices, while $\mathbb{H}_m^{++}$ and $\mathbb{H}_m^+$ denote the sets of $m \times m$ Hermitian positive definite and Hermitian positive semidefinite matrices, respectively. $[a]$ denotes the greatest integer less than or equal to a .

2. Preliminaries. The following lemmas are shown before we deduce the chief conclusions.

LEMMA 2.1. [27] For $M \in \mathbb{C}^{n \times n}$, $\rho(M) < 1$ if and only if there exists a nonsingular matrix $D \in \mathbb{C}^{n \times n}$ such that

$$\sigma_1(DMD^{-1}) < 1.$$

LEMMA 2.2. [22] Let $\{M_k\} \subset \mathbb{C}^{m \times m}$ be an infinite sequence of matrices. The series $\sum_{k=0}^{\infty} M_k$ converges to a matrix in $\mathbb{C}^{m \times m}$ if there exists a matrix norm $\|\cdot\|$ on the complex matrix space such that the associated numerical series $\sum_{k=0}^{\infty} \|M_k\|$ converges (or if its partial sums remain bounded).

LEMMA 2.3. [21] If $M_k(t) \in \mathbb{C}^{m \times m}$ for $k \geq 1$ are differentiable, then

$$\begin{aligned} \frac{d[M_1(t)M_2(t) \cdots M_k(t)]}{dt} &= \left[\frac{d}{dt} M_1(t) \right] M_2(t) \cdots M_k(t) \\ &+ X_1(t) \left[\frac{d}{dt} M_2(t) \right] M_3(t) \cdots M_k(t) + \cdots + M_1(t) \cdots M_{k-1} \left[\frac{d}{dt} M_k(t) \right]. \end{aligned}$$

LEMMA 2.4. [21] Let $M \in \mathbb{C}^{m \times m}$, then

$$\frac{d}{dt} e^{Mt} = M e^{Mt} = e^{Mt} M.$$

LEMMA 2.5. [22] Let $M \in \mathbb{C}^{m \times m}$, for any matrix norm, then

$$\rho(M) = \lim_{k \rightarrow \infty} \|M^k\|^{\frac{1}{k}}.$$

LEMMA 2.6. [22] Let $M \in \mathbb{C}^{m \times m}$ and $\|\cdot\|$ be a unitarily invariant matrix norm, then

$$\|M\|_2 \leq \|M\|.$$

Remark 2.7. Specifically, the Frobenius norm is the common unitarily invariant matrix norm.

3. An Iterative Algorithm for Solving the Riccati Matrix Differential Equation. In this section, we first prove that a convergent matrix series is the solution of a continuous algebraic Lyapunov matrix equation (CALE). Then, by using the sum of the first l items for the above convergent matrix series, we obtain a monotonically increasing lower bound sequence, and its limit is the solution of the above CALE. By using the above convergent matrix series, we can derive the new solution expression for the RDE (1.1). Finally, the corresponding iterative algorithm based on a matrix series for solving the RDE (1.1) and its convergence analysis are given.

THEOREM 3.1. Suppose that $A \in \mathbb{C}^{m \times m}$ is stable for the RDE (1.1), we have the following results:

(i) For any positive number q , the following matrix series

$$(3.1) \quad \sum_{k=1}^{\infty} [(qI_m + A)(qI_m - A)^{-1}]^{k-1} \cdot [2q(qI_m - A)^{-1}BR^{-1}B^*(qI_m - A)^{-*}] \cdot [(qI_m - A)^{-*}(qI_m + A)^*]^{k-1},$$

is convergent and is the solution of the following CALE:

$$(3.2) \quad AP + PA^* + BR^{-1}B^* = 0,$$

which is denoted as P_* .

(ii) For any positive integer l , then let us denote

$$(3.3) \quad S_l = \sum_{k=1}^l [(qI_m + A)(qI_m - A)^{-1}]^{k-1} \cdot [2q(qI_m - A)^{-1}BR^{-1}B^*(qI_m - A)^{-*}] \cdot [(qI_m - A)^{-*}(qI_m + A)^*]^{k-1},$$

then for arbitrary integer $n = 1, 2, \dots$, and any positive number q , we have the following results:

$$(3.4) \quad \begin{aligned} \widehat{P}_l^{(n)} &= [(qI_m + A)(qI_m - A)^{-1}]^n S_l [(qI_m - A)^{-*}(qI_m + A)^*]^n \\ &+ \sum_{k=1}^n [(qI_m + A)(qI_m - A)^{-1}]^{k-1} \cdot [2q(qI_m - A)^{-1}BR^{-1}B^*(qI_m - A)^{-*}] \\ &\cdot [(qI_m - A)^{-*}(qI_m + A)^*]^{k-1} \preceq P. \end{aligned}$$

Moreover, the monotonically increasing lower bound sequence $\{\widehat{P}_l^{(n)}\}$ converges, and there exists a matrix $\widehat{P}_{l_\infty}$ such that

$$\widehat{P}_{l_\infty} = \lim_{n \rightarrow \infty} \widehat{P}_l^{(n)},$$

the limit $\widehat{P}_{l_\infty}$ of the matrix sequence $\{\widehat{P}_l^{(n)}\}$ is equal to the solution P of the CALE (3.2).

(iii) Let $P_f \in \mathbb{H}_m^{++}$, then we can get the following matrix

$$e^{-A(t_f-t)}(P_f^{-1} + P_*)e^{-A^*(t_f-t)} - P_*,$$

which is Hermitian positive definite, and the RDE (1.1) has the Hermitian positive definite solution $P(t)$, which has the following form:

$$(3.5) \quad P(t) = \left[e^{-A(t_f-t)}(P_f^{-1} + P_*)e^{-A^*(t_f-t)} - P_* \right]^{-1}.$$

Proof. (i) For any positive number q , suppose that

$$V = qI_m - A,$$

due to A is stable, then we have

$$(qI_m + A)(qI_m - A)^{-1} = (2qI_m - V)V^{-1} = 2qV^{-1} - I_m.$$

Hence, for $j = 1, 2, \dots, m$, we can obtain

$$\begin{aligned} |\lambda_j [(qI_m + A)(qI_m - A)^{-1}]|^2 &= \left| 2q\lambda_j(V^{-1}) - 1 \right|^2 = \left| 2q\frac{1}{\lambda_{n-j+1}(V)} - 1 \right|^2 = \left| 2q\frac{1}{q - \lambda_j(A)} - 1 \right|^2 \\ &= \left| \frac{q + \lambda_j(A)}{q - \lambda_j(A)} \right|^2 = \frac{q + \lambda_j(A)}{q - \lambda_j(A)} \cdot \frac{q + \bar{\lambda}_j(A)}{q - \bar{\lambda}_j(A)} \\ &= \frac{q^2 + 2q\operatorname{Re}\lambda_j(A) + |\lambda_j(A)|^2}{q^2 - 2q\operatorname{Re}\lambda_j(A) + |\lambda_j(A)|^2} < 1, \end{aligned}$$

it follows that $\rho[(qI_m + A)(qI_m - A)^{-1}] < 1$.

Next, let us denote $\hat{A} = (qI_m + A)(qI_m - A)^{-1}$, $\hat{Q} = 2q(qI_m - A)^{-1}BR^{-1}B^*(qI_m - A)^{-*}$, $\tilde{A} = D\hat{A}D^{-1}$ and $\tilde{Q} = D\hat{Q}D^*$. Hence, from Lemma 2.1, we can find a nonsingular matrix D such that $\sigma_1(D\hat{A}D^{-1}) < 1$. Then, we define $\tilde{S}_n = \sum_{k=1}^n \|\tilde{A}^{k-1}\tilde{Q}(\tilde{A}^*)^{k-1}\|_2$ as the partial sum for the infinite norm series $\sum_{k=1}^{\infty} \|\tilde{A}^{k-1}\tilde{Q}(\tilde{A}^*)^{k-1}\|_2$.

Subsequently, we can get

$$\begin{aligned} \tilde{S}_n &= \sum_{k=1}^n \left\| \tilde{A}^{k-1}\tilde{Q}(\tilde{A}^*)^{k-1} \right\|_2 = \sum_{k=1}^n \sigma_1 \left[\tilde{A}^{k-1}\tilde{Q}(\tilde{A}^*)^{k-1} \right] \\ &= \sum_{k=1}^n \left| \lambda_1[\tilde{A}^{k-1}\tilde{Q}(\tilde{A}^*)^{k-1}] \right| \sum_{k=1}^n \left| \lambda_1(\tilde{Q})\lambda_1^{k-1}(\tilde{A}\tilde{A}^*) \right| \\ &= \lambda_1(\tilde{Q}) \cdot \sum_{k=1}^n \lambda_1^{k-1}(\tilde{A}\tilde{A}^*) \leq \lambda_1(\tilde{Q}) \cdot \frac{1 - \lambda_1^{n-1}(\tilde{A}\tilde{A}^*)}{1 - \lambda_1(\tilde{A}\tilde{A}^*)} \\ &\leq \frac{\lambda_1(\tilde{Q})}{1 - \lambda_1(\tilde{A}\tilde{A}^*)}. \end{aligned} \tag{3.6}$$

From (3.6), it is evident that $\tilde{S}_n = \sum_{k=1}^n \|\tilde{A}^{k-1}\tilde{Q}(\tilde{A}^*)^{k-1}\|_2$ is bounded, as stated in Lemma 2.2, which guarantees the series $\sum_{k=1}^{\infty} \tilde{A}^{k-1}\tilde{Q}(\tilde{A}^*)^{k-1}$ converges to a matrix, which is denoted as $\tilde{P}_*$.

Consequently, premultiplying by D^{-1} and postmultiplying by D^{-*} , we can get

$$\begin{aligned} D^{-1}\tilde{P}_*D^{-*} &= D^{-1} \sum_{k=1}^{\infty} (D\hat{A}D^{-1})^{k-1} D\hat{Q}D^* (D^{-*}\hat{A}^*D^*)^{k-1} D^{-*} \\ &= \sum_{k=1}^{\infty} \hat{A}^{k-1} D^{-1} D\hat{Q}D^* D^{-*} (\hat{A}^*)^{k-1} \\ &= \sum_{k=1}^{\infty} \hat{A}^{k-1} \hat{Q}(\hat{A}^*)^{k-1}, \end{aligned}$$

which is denote as P_* .

Due to (3.2) can be equivalently transformed into the following equation

$$(3.7) \quad P = \widehat{A}P\widehat{A}^* + \widehat{Q}.$$

Substituting $P_\star$ for P in (3.7),

$$\begin{aligned} \widehat{A}P_\star\widehat{A}^* + \widehat{Q} &= \widehat{A}\left[\sum_{k=1}^{\infty} \widehat{A}^{k-1}\widehat{Q}(\widehat{A}^*)^{k-1}\right]\widehat{A}^* + \widehat{Q} \\ &= \sum_{k=2}^{\infty} \widehat{A}^{k-1}\widehat{Q}(\widehat{A}^*)^{k-1} + \widehat{Q} = \sum_{k=1}^{\infty} \widehat{A}^{k-1}\widehat{Q}(\widehat{A}^*)^{k-1} = P_\star, \end{aligned}$$

then, we can obtain

$$P_\star = P = \sum_{k=1}^{\infty} \widehat{A}^{k-1}\widehat{Q}(\widehat{A}^*)^{k-1}.$$

(ii) For any positive number q , let us denote $\widehat{A} = (qI_m + A)(qI_m - A)^{-1}$, $\widehat{Q} = 2q(qI_m - A)^{-1}BR^{-1}B^*(qI_m - A)^{-*}$, for any natural number l , we denote $S_l = \sum_{k=1}^l \widehat{A}^{k-1}\widehat{Q}(\widehat{A}^*)^{k-1}$, from (i) it is obvious that $S_l \preceq P_\star$. Subsequently, we can transform (3.7) into the following equation, we have

$$P \succeq \widehat{A}S_l\widehat{A}^* + \widehat{Q} \equiv \widehat{P}_l^{(1)}.$$

Then, we can obtain

$$\begin{aligned} P &\succeq \widehat{A}[\widehat{A}S_l\widehat{A}^* + \widehat{Q}]\widehat{A}^* + \widehat{Q} \\ &= \widehat{A}^2S_l(\widehat{A}^*)^2 + \sum_{k=1}^2 \widehat{A}^{k-1}\widehat{Q}(\widehat{A}^*)^{k-1} \equiv \widehat{P}_l^{(2)}. \end{aligned}$$

By the recursive method, we have

$$(3.8) \quad P \succeq \widehat{A}^n S_l (\widehat{A}^*)^n + \sum_{k=1}^n \widehat{A}^{k-1} \widehat{Q} (\widehat{A}^*)^{k-1} \equiv \widehat{P}_l^{(n)}.$$

From the above analysis and by the recursive method, we can get

$$\widehat{P}_l^{(n+1)} = \widehat{A}\widehat{P}_l^{(n)}\widehat{A}^* + \widehat{Q},$$

for $n = 1$, we can obtain that

$$\widehat{P}_l^{(2)} - \widehat{P}_l^{(1)} = \widehat{A}[\widehat{A}S_l\widehat{A}^* + \widehat{Q} - S_l]\widehat{A}^* = \widehat{A}^l\widehat{Q}(\widehat{A}^*)^l \succeq 0.$$

This implies that it monotonically holds for $n = 1$. Then it is assumed that monotonically holds for $n = i - 1$, i.e. $\widehat{P}_l^{(i)} \succeq \widehat{P}_l^{(i-1)}$. With the induction assumption above, when $n = i$, we have

$$\widehat{P}_l^{(i+1)} - \widehat{P}_l^{(i)} = \widehat{A}[\widehat{P}_l^{(i)} - \widehat{P}_l^{(i-1)}]\widehat{A}^* + \widehat{Q} - \widehat{Q} \succeq 0.$$

206 Hence, we have

207 (3.9) $\hat{P}_l^{(i+1)} \succeq \hat{P}_l^{(i)}.$

According to (3.8) and (3.9), we can know that the sequence of Hermitian positive semidefinite matrix sequence $\{\hat{P}_l^{(n)}\}_{n=1}^\infty$ is convergent, hence, there exists $\hat{P}_{l_\infty} \in \mathbb{H}_m^+$ such that

$$\hat{P}_{l_\infty} = \lim_{n \rightarrow \infty} \hat{P}_l^{(n)} = \sum_{k=1}^{\infty} \hat{A}^{k-1} \hat{Q} (\hat{A}^*)^{k-1}.$$

208 Similar to the proof of (i), we can get

209 $\hat{P}_{l_\infty} = P = P_\star.$

210 (iii) Since $P_f \in \mathbb{H}_m^{++}$, hence $P_f^{-1} \in \mathbb{H}_m^{++}$, let us consider the following matrix differential
211 equation:

212 (3.10)
$$\begin{cases} \dot{X}(t) = AX(t) + X(t)A^* - BR^{-1}B^*, & t \in [t_0, t_f], \\ X(t_f) = P_f^{-1}, \end{cases}$$

On the hand, from (3.10), and multiplying the equation by e^{-At} on the left and e^{-A^*t} on the right, we can get

$$e^{-At} \left\{ \frac{d[X(t)]}{dt} - AX(t) - X(t)A^* \right\} e^{-A^*t} = -e^{-At} BR^{-1}B^* e^{-A^*t}$$

Similarly, substituting t with s and performing a definite integral on both sides of the equation, and the integral interval is from t to t_f , which yields the following equation:

$$\int_t^{t_f} \frac{d[e^{-As}X(s)e^{-A^*s}]}{ds} ds = - \int_t^{t_f} e^{-As} BR^{-1}B^* e^{-A^*s} ds.$$

213 Then, we can obtain another expression for $X(t)$

214
$$X(t) = e^{-A(t_f-t)} P_f^{-1} e^{-A^*(t_f-t)} + \int_t^{t_f} e^{-A(s-t)} BR^{-1}B^* e^{-A^*(s-t)} ds,$$

due to $P_f^{-1} \in \mathbb{H}_m^{++}$ and $BR^{-1}B^* \in \mathbb{H}_m^+$, we can get

$$X(t) = e^{-A(t_f-t)} P_f^{-1} e^{-A^*(t_f-t)} + \int_t^{t_f} e^{-A(s-t)} BR^{-1}B^* e^{-A^*(s-t)} ds$$

215 is Hermitian positive definite,

216 On the other hand, the (3.10) can be transformed into the following equivalent form:

217
$$\dot{X}(t) = A[X(t) + P_\star] + [X(t) + P_\star]A^* - AP_\star - P_\star A^* - BR^{-1}B^*,$$

due to (i), and multiplying the equation by e^{-At} on the left and e^{-A^*t} on the right, we can get

$$e^{-At} \{ \dot{X}(t) - A[X(t) + P_\star] - [X(t) + P_\star]A^* \} e^{A^*t} = 0,$$

Then, substituting t with s and performing a definite integral on both sides of the equation, the integral interval is from t to t_f , which yields the following equation:

$$\int_t^{t_f} \frac{d\{e^{-As}[X(s) + P_\star]e^{-A^*s}\}}{ds} = 0,$$

i.e.,

$$X(t) = e^{-A(t_f-t)}[X(t_f) + P_\star]e^{-A^*(t_f-t)} - P_\star.$$

Next, we prove the $X^{-1}(t)$ with the terminal condition $X^{-1}(t_f) = P_f$ satisfies the RDE (1.1).

Due to $\dot{X}^{-1}(t) = -X^{-1}(t)\dot{X}(t)X^{-1}(t)$, then according to (i), we can obtain

$$\begin{aligned} -\dot{X}^{-1}(t) &= X^{-1}(t) \left(A e^{-A(t_f-t)} (P_f^{-1} + P_\star) e^{-A^*(t_f-t)} + e^{-A(t_f-t)} (P_f^{-1} + P_\star) e^{-A^*(t_f-t)} A^* \right) X^{-1}(t) \\ &= X^{-1}(t) \left\{ A [e^{-A(t_f-t)} (P_f^{-1} + P_\star) e^{-A^*(t_f-t)} - P_\star] \right. \\ &\quad \left. + A P_\star + [e^{-A(t_f-t)} (P_f^{-1} + P_\star) e^{-A^*(t_f-t)} - P_{t_\infty}] A^* + P_\star A^* \right\} X^{-1}(t) \\ &= X^{-1}(t) A X(t) X^{-1}(t) + X^{-1}(t) X(t) A^* X^{-1}(t) + X^{-1}(t) (A P_\star + P_\star A^* + B R^{-1} B^*) X^{-1}(t) \\ &\quad - X^{-1}(t) B R^{-1} B^* X^{-1}(t) \\ &= A^* X^{-1}(t) + X^{-1}(t) A - X^{-1}(t) B R^{-1} B^* X^{-1}(t), \end{aligned}$$

with the terminal condition $X^{-1}(t_f) = P_f \in \mathbb{H}_m^{++}$.

To sum, according to the above analysis, we can know that

$$X(t) = e^{-A(t_f-t)}[X(t_f) + P_\star]e^{-A^*(t_f-t)} - P_\star$$

is Hermitian positive definite, and

$$P(t) = X^{-1}(t) = [e^{-A(t_f-t)} (P_f^{-1} + P_\star) e^{-A^*(t_f-t)} - P_\star]^{-1},$$

with the terminal condition $P(t_f) = X^{-1}(t_f) = P_f \in \mathbb{H}_m^{++}$ is the Hermitian positive definite solution for the RDE (1.1).

The proof is completed. $\square$

Remark 3.2. Generally, for the convenience of calculation, we usually take $S_1 = 2q(qI_m - A)^{-1} B R^{-1} B^* (qI_m - A)^{-*}$.

In [17, 14], the authors pointed out that when we can not get the exact solution in actual implementation, we often use the norm of the difference between the iteration values between two steps as less than the tolerance ϵ as the stopping criterion. Based on this reason, the stopping criteria for Algorithm 1 as follows:

$$\|\hat{P}_l^{(n+1)} - \hat{P}_l^{(n)}\|_2 < \epsilon.$$

Subsequently, according to Theorem 3.1, we give the following algorithm for solving RDE (1.1) based on the lower increasing sequence.

Algorithm 1

Input: The initial and final times t_0, t_f , the stable matrix $A \in \mathbb{C}^{m \times m}$, the matrix $B \in \mathbb{C}^{m \times p}$, the matrices $R \in \mathbb{H}_p^+$, $P_f \in \mathbb{H}_m^{++}$, I_m , the number N of nodes, the step time $\delta_T = (t_f - t_0)/N$, the proper parameter q , and the tolerance ϵ .

1: Compute $\hat{A} = (qI_m + A)(qI_m - A)^{-1}$, $\hat{Q} = 2q(qI_m - A)^{-1}BR^{-1}B^*(qI_m - A)^{-*}$, $S_l = \sum_{k=1}^l \hat{A}^{k-1}\hat{Q}(\hat{A}^*)^{k-1}$.

2: Compute $\hat{P}_l^{(1)} = \hat{A}S_l\hat{A}^* + \hat{Q}$ and $\hat{P}_l^{(2)} = \hat{A}\hat{P}_l^{(1)}\hat{A}^* + \hat{Q}$.

234 3: If $\|\hat{P}_l^{(n+1)} - \hat{P}_l^{(n)}\| < \epsilon$, then run Step 6-9; else, go to the next step.

4: $\hat{P}_l^{(n+1)} = \hat{A}\hat{P}_l^{(n)}\hat{A}^* + \hat{Q}$; $n = n + 1$.

5: Go to Step 3.

6: **For** $j = 1, \dots, N$ **do**

7: Compute: $t_j = (j - 1) \cdot \delta_T$;

8: Compute: $P_a(t_j) = \left[e^{-A(t_f - t_j)}(P_f^{-1} + \hat{P}_l^{(n+1)})e^{-A^*(t_f - t_j)} - \hat{P}_l^{(n+1)} \right]^{-1}$.

9: **End for**

Output: $P_a(t_j)$, $1 \leq j \leq N$.

235 *Remark 3.3.* Compared with classical approaches for solving the CALE, such as the Bar-
 236 tels–Stewart algorithm, the proposed matrix series method for obtaining $P_\star$ demonstrates clear
 237 advantages under limited computational resources and sparse coefficient matrices. Specifically,
 238 each term in the series $\hat{A}^k\hat{Q}(\hat{A}^*)^k$ can be computed efficiently through sparse matrix multipli-
 239 cations, which only store and process non-zero elements, thereby significantly reducing memory
 240 usage. Moreover, the computation can be terminated early once the desired approximation ac-
 241 curacy is achieved. In contrast, traditional algorithms like Bartels–Stewart rely on dense matrix
 242 factorizations (e.g., Schur decomposition), which fail to exploit sparsity and entail $O(n^3)$ time and
 243 $O(n^2)$ memory complexities. These methods require all computations to be performed in a sin-
 244 gle step and do not yield intermediate results, limiting their applicability in real-time scenarios.
 245 Furthermore, when computational resources are ample, the proposed approach also benefits from
 246 potential parallelization, offering additional improvements in efficiency.

247 *Remark 3.4.* In steps 6 to 9 of Algorithm 1, we improve efficiency by streamlining the computa-
 248 tion of matrix powers. Outside the loop, we precompute $e^{A\delta_T}$, calculate $P_{\text{diff}} = P_f^{-1} + \hat{P}_l^{(n+1)}$, and
 249 initialize $M_0 = e^{-At_f}$. Within the loop ($j = 1, \dots, N$), each iteration computes the matrix power
 250 iteratively as $M_j = M_{j-1} \cdot e^{A\delta_T}$, determines the conjugate transpose $M_j^* = (M_j)^*$, and computes
 251 $P_a(t_j) = M_j \cdot P_{\text{diff}} \cdot M_j^* - \hat{P}_l^{(n+1)}$. By iteratively calculating matrix powers, this method avoids the
 252 direct computation of $e^{A(-t_f + j\delta_T)}$, substantially lowering computational complexity. This approach
 253 can also be extended to the corresponding steps in Algorithm 2.

254 The convergence of this method is established in Theorem 3.1. Consequently, the efficiency
 255 of computing the approximate solution $P_a(t) = \left[e^{-A(t_f - t)}(P_f^{-1} + \hat{P}_l^{(n+1)})e^{-A^*(t_f - t)} - \hat{P}_l^{(n+1)} \right]^{-1}$
 256 depends on the convergence rate of the matrix sequence $\{\hat{P}_l^{(n)}\}$. In the subsequent remark, we
 257 analyze the convergence rate of $\{\hat{P}_l^{(n)}\}$ to further elucidate this dependency.

258 *Remark 3.5.* The matrix sequence $\{\hat{P}_l^{(n)}\}$ generated by (3.4) monotonically increasing and
 259 converges to $P_\star$. Moreover, the convergent rate is linear, with an R_1 -convergence factor of at most

260 $\rho^2[(qI_m + A)(qI_m - A)^{-1}]$. Specifically, it holds that

$$261 \quad \limsup_{n \rightarrow \infty} \sqrt[n]{\|P_\star - \hat{P}_l^{(n)}\|} \leq \rho^2[(qI_m + A)(qI_m - A)^{-1}].$$

262 *Proof.* For given any positive number q , let us first denote $\hat{A} = (qI_m + A)(qI_m - A)^{-1}$ and
 263 $\hat{Q} = 2q(qI_m - A)^{-1}BR^{-1}B^*(qI_m - A)^{-*}$, from Theorem 3.1, we have

$$264 \quad P_\star = \sum_{k=1}^{\infty} \hat{A}^{k-1} \hat{Q} (\hat{A}^*)^{k-1},$$

265 and

$$266 \quad \hat{P}_l^{(n)} = \hat{A}^n S_l (\hat{A}^*)^n + \sum_{k=1}^n \hat{A}^{k-1} \hat{Q} (\hat{A}^*)^{k-1}.$$

267 Thus,

$$268 \quad P_\star - \hat{P}_l^{(n)} = \sum_{k=n+1}^{\infty} \hat{A}^{k-1} \hat{Q} (\hat{A}^*)^{k-1} - \hat{A}^n S_l (\hat{A}^*)^n$$

$$269 \quad = \hat{A}^n \cdot \left[\sum_{k=1}^{\infty} \hat{A}^{k-1} \hat{Q} (\hat{A}^*)^{k-1} - S_l \right] \cdot (\hat{A}^*)^n.$$

270 Due to $\sum_{k=1}^{\infty} \hat{A}^{k-1} \hat{Q} (\hat{A}^*)^{k-1}$ is convergent from Theorem 3.1 and S_l is lower matrix bounds
 271 for the solution of (3.1), then taking the norm of both sides and using the property that, for any
 272 matrix norm on $\mathbb{C}^{m \times m}$, $\lim_{n \rightarrow \infty} \sqrt[n]{\|\hat{A}^n\|} = \rho(\hat{A}) < 1$, we obtain

$$273 \quad \limsup_{n \rightarrow \infty} \sqrt[n]{\|P_\star - \hat{P}_l^{(n)}\|} = \limsup_{n \rightarrow \infty} \sqrt[n]{\hat{A}^n \cdot \left[\sum_{k=1}^{\infty} \hat{A}^{k-1} \hat{Q} (\hat{A}^*)^{k-1} - S_l \right] \cdot (\hat{A}^*)^n}$$

$$274 \quad \leq \rho(\hat{A}) \cdot \rho(\hat{A}^*) = \rho^2(\hat{A}) < 1.$$

275 Hence, the convergence rate of the matrix sequence $\{\hat{P}_l^{(n)}\}$ is at least linear, with an R_1 -convergence
 276 factor bounded above by $\rho^2(\hat{A})$. $\square$

277 As is widely recognized, the parameter q influences the convergence of the algorithm. Thus, at
 278 the end of this section, we focus on discussing a convenient method for selecting an appropriate q .

279 *Remark 3.6.* From Theorem 3.1, the numerical solution $P_a(t)$ of the RDE (1.1) depends on the
 280 convergent rate matrix sequence $\{\hat{P}_l^{(n)}\}$. According to Remark 3.5, a smaller $\rho[(qI_m + A)(qI_m -$
 281 $A)^{-1}]$ yields a tighter upper bound for $\limsup_{n \rightarrow \infty} \sqrt[n]{\|P_\star - \hat{P}_l^{(n)}\|}$, resulting in a more accurate
 282 $\hat{P}_l^{(n)}$ and a faster convergence rate. Furthermore, $\rho[(qI_m + A)(qI_m - A)^{-1}]$ is given by

$$283 \quad \rho[(qI_m + A)(qI_m - A)^{-1}] = \max_{1 \leq j \leq m} |\lambda_j[(qI_m + A)(qI_m - A)^{-1}]|$$

$$284 \quad (3.11) \quad = \max_{1 \leq j \leq m} \left\{ \sqrt{\frac{q^2 + 2q \operatorname{Re} \lambda_j(A) + |\lambda_j(A)|^2}{q^2 - 2q \operatorname{Re} \lambda_j(A) + |\lambda_j(A)|^2}} \right\},$$

where $|\lambda_j(A)|^2 = \operatorname{Re} \lambda_j(A)^2 + \operatorname{Im} \lambda_j(A)^2$. Clearly, $\rho(\hat{A})$ depends on the parameter q and the real and imaginary parts of the eigenvalues of A , making the selection of q a critical consideration.

When the eigenvalues of the stable matrix $A \in \mathbb{C}^{m \times m}$ are negative real numbers and $q > 0$, (3.11) simplifies to

$$\rho[(qI_m + A)(qI_m - A)^{-1}] = \max_{1 \leq j \leq m} \left\{ \frac{|q + \lambda_j(A)|}{q - \lambda_j(A)} \right\}.$$

We consider two cases:

(i) If $\lambda_1(A) = \lambda_m(A)$ and $q > 0$, then

$$\rho[(qI_m + A)(qI_m - A)^{-1}] = \frac{|q + \lambda_1(A)|}{q - \lambda_1(A)} = \frac{|q + \lambda_m(A)|}{q - \lambda_m(A)}.$$

(ii) If $\lambda_1(A) \neq \lambda_m(A)$ and $q > 0$, define the function

$$f(c) = \frac{|c + q|}{-c + q}, \quad (c < 0).$$

For $c < -q$, $f(c) = \frac{-(c+q)}{-c+q}$, and

$$f'(c) = \frac{-(-c+q) - (c+q)}{(-c+q)^2} = \frac{-2q}{(-c+q)^2} < 0,$$

indicating $f(c)$ is decreasing. For $-q < c < 0$, $f(c) = \frac{c+q}{-c+q}$, and

$$f'(c) = \frac{(-c+q) + (c+q)}{(-c+q)^2} = \frac{2q}{(-c+q)^2} > 0,$$

indicating $f(c)$ is increasing. Thus, $f(c)$ achieves its maximum at $c = \lambda_1(A)$ or $c = \lambda_m(A)$, implying $\rho[(qI_m + A)(qI_m - A)^{-1}]$ is determined by either $\lambda_1(A)$ or $\lambda_m(A)$.

Furthermore, when $q = \sqrt{\lambda_1(A)\lambda_m(A)}$, then we can obtain

$$f(\lambda_1(A))|_{q=\sqrt{\lambda_1(A)\lambda_m(A)}} = f(\lambda_m(A))|_{q=\sqrt{\lambda_1(A)\lambda_m(A)}} = \frac{-\sqrt{-\lambda_1(A)} + \sqrt{-\lambda_m(A)}}{\sqrt{-\lambda_m(A)} + \sqrt{-\lambda_1(A)}}.$$

Moreover, when $q \neq \sqrt{\lambda_1(A)\lambda_m(A)}$, we have the following results:

$$\begin{aligned} & f(\lambda_1(A))|_{q>\sqrt{\lambda_1(A)\lambda_m(A)}} - f(\lambda_1(A))|_{q=\sqrt{\lambda_1(A)\lambda_m(A)}} \\ & > \frac{2(\sqrt{\lambda_1(A)\lambda_m(A)}\sqrt{-\lambda_1(A)} + \lambda_1(A)\sqrt{-\lambda_m(A)})}{(q - \lambda_1(A))(\sqrt{-\lambda_m(A)} + \sqrt{-\lambda_1(A)})} = 0, \end{aligned} \quad (3.12)$$

and

$$\begin{aligned} & f(\lambda_m(A))|_{q<\sqrt{\lambda_1(A)\lambda_m(A)}} - f(\lambda_m(A))|_{q=\sqrt{\lambda_1(A)\lambda_m(A)}} \\ & > \frac{-2(\sqrt{\lambda_1(A)\lambda_m(A)}\sqrt{-\lambda_m(A)} + \lambda_m(A)\sqrt{-\lambda_1(A)})}{(q - \lambda_m(A))(\sqrt{-\lambda_m(A)} + \sqrt{-\lambda_1(A)})} = 0, \end{aligned} \quad (3.13)$$

hence, we can easily get when we choose $q = \sqrt{\lambda_1(A)\lambda_m(A)}$, the value of $\rho(\hat{A})|_{q \in (0, +\infty)}$ will achieve the minimal value.

However, when A has complex eigenvalues, determining which $\lambda_j(A)$ maximizes $\rho[(qI_m + A)(qI_m - A)^{-1}]$ is nontrivial. For example, consider A with eigenvalues $\{-1+2i, -3+9i, -5+3i\}$ and $q = 1$. Computing (3.11) yields $|\lambda_1[(qI_m + A)(qI_m - A)^{-1}]| = 0.7071$, $|\lambda_2[(qI_m + A)(qI_m - A)^{-1}]| = 0.9361$, and $|\lambda_3[(qI_m + A)(qI_m - A)^{-1}]| = 0.7454$, so $\rho[(qI_m + A)(qI_m - A)^{-1}] = 0.9361$ at $\lambda_2(A)$. This illustrates the difficulty in predicting the maximizing eigenvalue for complex cases.

Drawing from the real eigenvalue case, we propose $q = \sqrt{\lambda_1(A)\lambda_m(A)}$ and validate it through numerical experiments. The results suggest improved convergence rates for the numerical solution. Thus, in this paper's algorithms, we adopt the optimal parameter as follows:

$$q_{opt} = \sqrt{\max_{j=1,2,\dots,m} |\lambda_j(A)| \cdot \min_{j=1,2,\dots,m} |\lambda_j(A)|}.$$

Remark 3.7. In this study, we employ MATLAB's 'eig' function to calculate the eigenvalues of matrix A or to determine the spectral radius $\rho(\hat{A})$. Alternatively, the power iteration method can be applied to compute these values, offering reduced computational complexity and shorter CPU processing time.

4. An Accelerated Iterative Algorithm for Solving the Riccati Matrix Differential Equation. In this section, we aim to compute $\hat{P}_l^{(n+1)}$ more efficiently, reduce the computational complexity of Algorithm 1, and enhance Algorithm 1's efficiency at a fundamental level while preserving its precision. To derive the main conclusion in this section, we first present a detailed analysis of the error for the proposed method in Theorem 3.1.

Next, to provide an error analysis of this method, let us first denote $\hat{A} = (qI_m + A)(qI_m - A)^{-1}$ and $\hat{Q} = 2q(qI_m - A)^{-1}BR^{-1}B^*(qI_m - A)^{-*}$, then provide the following symbol definitions and explanations. Suppose $E_\star$ denotes the difference between the matrix series

$$(4.1) \quad P_\star = \sum_{k=1}^{\infty} \hat{A}^{k-1} \hat{Q} (\hat{A}^*)^{k-1},$$

and the approximate matrix series

$$\hat{P}_l^{(n+1)} = \hat{A}^{n+1} S_l(\hat{A}^*)^{n+1} + \sum_{k=1}^{n+1} \hat{A}^{k-1} \hat{Q} (\hat{A}^*)^{k-1}.$$

We then define $\hat{E}(t) = P^{-1}(t) - P_a^{-1}(t)$, where

$$(4.2) \quad P^{-1}(t) = e^{-A(t_f-t)}(P_f^{-1} + P_\star)e^{-A^*(t_f-t)} - P_\star,$$

and

$$(4.3) \quad P_a^{-1}(t) = e^{-A(t_f-t)} \left[P_f^{-1} + \hat{P}_l^{(n+1)} \right] e^{-A^*(t_f-t)} - \hat{P}_l^{(n+1)}.$$

And we define $E(t)$ as the difference between the exact solution $P(t)$ from (3.5) and the approximate solution

$$P_a(t) = \left[e^{-A(t_f-t)}(P_f^{-1} + \hat{P}_l^{(n+1)})e^{-A^*(t_f-t)} - \hat{P}_l^{(n+1)} \right]^{-1}$$

for the RDE (1.1).

Based on the definitions above, we illustrate the linear matrix equation satisfied by $\hat{E}(t)$ and establish upper bounds for the norms of the matrices E_* , $\hat{E}(t)$, and $E(t)$.

PROPOSITION 4.1. *The error matrix $\hat{E}(t)$ satisfies the following linear matrix equation:*

$$(4.4) \quad \hat{E}(t) = P^{-1}(t) - P_a^{-1}(t) = e^{-A(t_f-t)} [P_* - \hat{P}_l^{(n+1)}] e^{-A^*(t_f-t)} - [P_* - \hat{P}_l^{(n+1)}],$$

where P_* and $\hat{P}_l^{(n+1)}$ are defined as (4.1) and (4.2), respectively.

Proof. Subtracting equation (4.3) from equation (4.2) yields (4.4). $\square$

By Lemma 2.5, we have

$$(4.5) \quad \rho(\hat{A}) = \lim_{k \rightarrow \infty} \|\hat{A}^k\|^{\frac{1}{k}}.$$

From the proof of Theorem 3.1, since $\rho(\hat{A}) < 1$, then we define a positive constant $\xi = \frac{1-\rho(\hat{A})}{2} > 0$ such that $\rho(\hat{A}) + \xi < 1$. Using (4.5) and the definition of the limit, there exists a positive integer N_ξ such that, for $k \geq N_\xi$,

$$\left| \|\hat{A}^k\|^{\frac{1}{k}} - \rho(\hat{A}) \right| < \xi,$$

i.e.,

$$\|\hat{A}^k\| < [\rho(\hat{A}) + \xi]^k.$$

Similarly, for $k \geq N_\xi$,

$$\|(\hat{A}^*)^k\| < [\rho(\hat{A}) + \xi]^k.$$

Thus, for $k \geq N_\xi$,

$$\|\hat{A}^k \hat{Q} (\hat{A}^*)^k\| \leq \|\hat{A}^k\| \cdot \|\hat{Q}\| \cdot \|(\hat{A}^*)^k\| < \|\hat{Q}\| \cdot [\rho(\hat{A}) + \xi]^{2k}.$$

Since $\rho(\hat{A}) + \xi < 1$, let $\beta = [\rho(\hat{A}) + \xi]^2 < 1$, so $[\rho(\hat{A}) + \xi]^{2(k-1)} = \beta^{k-1} < 1$. Then, for $n \geq N_\xi$,

$$(4.6) \quad \begin{aligned} \|E_*\| &= \|P_* - \hat{P}_l^{(n+1)}\| = \left\| \sum_{k=n+2}^{\infty} \hat{A}^{k-1} \hat{Q} (\hat{A}^*)^{k-1} - \hat{A}^{n+1} S_l (\hat{A}^*)^{n+1} \right\| \\ &\leq \left\| \sum_{k=n+2}^{\infty} \hat{A}^{k-1} \hat{Q} (\hat{A}^*)^{k-1} \right\| \leq \|\hat{Q}\| \cdot \sum_{k=n+2}^{\infty} \beta^{k-1} = \frac{\beta^{n+1} \|\hat{Q}\|}{1 - \beta}. \end{aligned}$$

Given a desired error precision $\epsilon > 0$, we can derive an initial estimate for n such that $\|E_*\| < \epsilon$.

PROPOSITION 4.2. *For a given error precision $\epsilon > 0$, assuming $\|\hat{Q}\| \neq 0$ and $\frac{\epsilon(1-\beta)}{\|\hat{Q}\|} < \beta$, an initial estimate for the positive integer n is*

$$(4.7) \quad n = \left\lceil \frac{\ln \left(\frac{\epsilon(1-\beta)}{\|\hat{Q}\|} \right)}{\ln \beta} \right\rceil,$$

where $\hat{A} = (qI_m + A)(qI_m - A)^{-1}$, $\hat{Q} = 2q(qI_m - A)^{-1}BR^{-1}B^*(qI_m - A)^{-*}$, $\beta = [\rho(\hat{A}) + \xi]^2$, and $\xi = \frac{1-\rho(\hat{A})}{2}$.

Proof. From the previous analysis, for a given $\epsilon > 0$ satisfying

$$\frac{\beta^{n+1} \|\widehat{Q}\|}{1 - \beta} < \epsilon,$$

implying

$$\beta^{n+1} < \frac{\epsilon(1 - \beta)}{\|\widehat{Q}\|}.$$

Taking the natural logarithm and noting that $0 < \beta < 1$, we obtain

$$n + 1 > \frac{\ln\left(\frac{\epsilon(1 - \beta)}{\|\widehat{Q}\|}\right)}{\ln \beta}.$$

Thus, an initial estimate is

$$n = \left\lceil \frac{\ln\left(\frac{\epsilon(1 - \beta)}{\|\widehat{Q}\|}\right)}{\ln \beta} \right\rceil.$$

The initial estimate (4.7) from Proposition 4.2 may not satisfy $\|\widehat{A}^n\| < [\rho(\widehat{A}) + \xi]^n$. The following remarks refine the determination of n .

Remark 4.3. If $A \in \mathbb{C}^{m \times m}$ is stable and Hermitian, then $qI_m + A$ and $(qI_m - A)^{-1}$ are Hermitian, since $(qI_m + A)(qI_m - A)^{-1} = (qI_m - A)^{-1}(qI_m + A)$. Thus, $\widehat{A} = (qI_m + A)(qI_m - A)^{-1}$ is Hermitian and $\rho(\widehat{A}) < 1$. Moreover, $\|(\widehat{A})^k\|_2 = \rho(\widehat{A}^k) = \rho^k(\widehat{A}) < [\rho(\widehat{A}) + \xi]^k$ for $k = 1, 2, \dots$. Using

$$n = \left\lceil \frac{\ln\left(\frac{\epsilon(1 - \beta)}{\|\widehat{Q}\|}\right)}{\ln \beta} \right\rceil \text{ ensures } \|E_\star\|_2 < \epsilon.$$

Remark 4.4. If $A \in \mathbb{C}^{m \times m}$ is stable and non-Hermitian, compute $n = \left\lceil \frac{\ln\left(\frac{\epsilon(1 - \beta)}{\|\widehat{Q}\|}\right)}{\ln \beta} \right\rceil$ from

Proposition 4.2 and verify if $\|(\widehat{A})^n\| < [\rho(\widehat{A}) + \xi]^n$. If not, incrementally increase n until the inequality holds, ensuring $\|E_\star\| \leq \epsilon$.

Remark 4.5. The bound $\|E_\star\| \leq \frac{\beta^{n+1} \|\widehat{Q}\|}{1 - \beta}$ holds for any matrix norm on $\mathbb{C}^{m \times m}$, allowing flexibility in choosing a norm for computing (4.7). For a non-Hermitian and stable matrix A and large-scale $\widehat{Q}$, the Frobenius norm $\|\cdot\|_F$ is preferable, as computing $\|\widehat{Q}\|_F$ requires only the diagonal elements of $\widehat{Q}\widehat{Q}^*$, which is faster than the spectral norm. However, for a Hermitian, stable A , per Lemma 2.6, the Frobenius norm may yield a larger n in (4.7), while the spectral norm may provide a more precise n , reducing additional computation. Thus, norm selection is critical for efficiency.

Next, we establish an upper bound for the norm of $\widehat{E}(t)$.

PROPOSITION 4.6. *The norm of $\widehat{E}(t)$ satisfies*

$$(4.8) \quad \|\widehat{E}(t)\| \leq [e^{(\|A\| + \|A^*\|)(t_f - t)} + 1] \cdot \|E_\star\|, \quad \forall t \leq t_f,$$

where $E_\star = P_\star - \widehat{P}_l^{(n+1)}$.

387 *Proof.* Since $\widehat{E}(t)$ satisfies (4.4) and $\|e^{sM}\| \leq e^{s\|M\|}$ for $s \geq 0$, for $t \leq t_f$,

$$\begin{aligned}
 388 \quad \|\widehat{E}(t)\| &\leq \|e^{-A(t_f-t)}\| \cdot \|E_\star\| \cdot \|e^{-A^*(t_f-t)}\| + \|E_\star\| \\
 389 \quad &\leq e^{\|A\|(t_f-t)} \cdot \|E_\star\| \cdot e^{\|A^*\|(t_f-t)} + \|E_\star\| \\
 390 \quad &= [e^{(\|A\|+\|A^*\|)(t_f-t)} + 1] \cdot \|E_\star\|.
 \end{aligned}$$

391 The proof is completed. $\square$

392 *Remark 4.7.* Once n is determined, an upper bound for $\|E_\star\|$ follows from (4.6), enabling an
 393 upper bound for $\|\widehat{E}(t)\|$ via (4.8).

394 Finally, we present an upper bound for the Frobenius norm of $E(t)$.

395 **PROPOSITION 4.8.** *The Frobenius norm of $E(t)$ satisfies*

$$396 \quad \|E(t)\|_F \leq \sigma_1[P(t)] \cdot \sigma_1[P_a(t)] \cdot \|\widehat{E}(t)\|_F, \quad \forall t \leq t_f.$$

397 *Proof.* From [12], for $B, C, Z \in \mathbb{C}^{m \times m}$, $\|BZC^*\|_F \leq \|B\|_2 \cdot \|C\|_2 \cdot \|Z\|_F$. Thus,

$$\begin{aligned}
 398 \quad \|E(t)\|_F &= \|P(t) - P_a(t)\|_F = \|P(t)(P_a^{-1}(t) - P^{-1}(t))P_a(t)\|_F \\
 399 \quad &\leq \|P(t)\|_2 \cdot \|P_a(t)\|_2 \cdot \|\widehat{E}(t)\|_F \\
 400 \quad &= \sigma_1[P(t)] \cdot \sigma_1[P_a(t)] \cdot \|\widehat{E}(t)\|_F.
 \end{aligned}$$

401 The proof is completed. $\square$

402 *Remark 4.9.* Replacing the Frobenius norm in Proposition 4.8 with the spectral norm preserves
 403 the validity of the bound.

404 *Remark 4.10.* An upper bound for $\|\widehat{E}(t)\|_F$ or $\|\widehat{E}(t)\|_2$ directly yields an upper bound for
 405 $\|E(t)\|_F$ or $\|E(t)\|_2$, respectively.

406 Notably, n can be expressed as $n = m_0 n_0 + q_0$, where m_0 , n_0 , and q_0 are natural numbers with
 407 $q_0 < n_0$. When n is prime, we require $q_0 > 0$; when n is composite, $q_0 \geq 0$ suffices. Leveraging
 408 properties of matrix series and powers, we propose a segmented computation method for $\widehat{P}_l^{(n+1)}$.
 409 The approach proceeds as follows:

410 First, compute $\widehat{A}$ and $(\widehat{A})^*$ and store them, then calculate $S_{q_0} = \sum_{k=1}^{q_0+1} \widehat{A}^{k-1} \widehat{Q}(\widehat{A}^*)^{k-1}$ using
 411 q_0 iterations with $\widehat{Q}$ as the initial value. Next, calculate $S_{q_0+2} - S_{q_0+1} = \widehat{A}^{q_0+1} \widehat{Q}(\widehat{A}^*)^{q_0+1}$. Then,
 412 perform $n_0 - 1$ iterations starting from $\widehat{A}^{q_0+1} \widehat{Q}(\widehat{A}^*)^{q_0+1}$ to obtain $\widehat{S}_{n_0} = \sum_{k=q_0+2}^{n_0+q_0+1} \widehat{A}^{k-1} \widehat{Q}(\widehat{A}^*)^{k-1}$.
 413 For the remaining terms, compute $\widehat{A}^{n_0}$ and $(\widehat{A}^{n_0})^*$ and store them, then perform $m_0 - 1$ itera-
 414 tions with $\widehat{A}^{n_0} \widehat{S}_{n_0} (\widehat{A}^*)^{n_0}$ as the initial value to yield $\widehat{S}_{(m_0-1)n_0} = \sum_{k=1}^{m_0} (\widehat{A}^{n_0})^{k-1} \widehat{S}_{n_0} \left[(\widehat{A}^*)^{n_0} \right]^{k-1}$.
 415 Moreover, $\widehat{A}^{n+1} S_l (\widehat{A}^*)^{n+1} = (\widehat{A}^{n_0})^{m_0} \cdot \widehat{A}^{q_0+1} \cdot S_l \cdot (\widehat{A}^{q_0+1})^* [(\widehat{A}^{n_0})^{m_0}]^*$. Hence, the total series

416 $\widehat{P}_l^{(n+1)} = \widehat{A}^{n+1} S_l(\widehat{A}^*)^{n+1} + \sum_{k=1}^{n+1} \widehat{A}^{k-1} \widehat{Q}(\widehat{A}^*)^{k-1}$ is then the sum of these segments:

$$\begin{aligned}
 417 \quad \widehat{P}_l^{(n+1)} &= \underbrace{\widehat{Q} + \widehat{A}\widehat{Q}(\widehat{A}^*) + \cdots + \widehat{A}^{q_0}\widehat{Q}(\widehat{A}^*)^{q_0}}_{\text{by } q_0 \text{ iterations}} \quad \begin{matrix} := \widehat{S}_{q_0} \\ \end{matrix} \\
 418 \quad &+ \underbrace{\widehat{A}^{q_0+1}\widehat{Q}(\widehat{A}^*)^{q_0+1} + \widehat{A}^{q_0+2}\widehat{Q}(\widehat{A}^*)^{q_0+2} + \cdots + \widehat{A}^{n_0+q_0}\widehat{Q}(\widehat{A}^*)^{n_0+q_0}}_{\text{by } n_0 - 1 \text{ iterations}} \quad \begin{matrix} := \widehat{S}_{n_0} \\ \end{matrix} \\
 419 \quad &+ \underbrace{\widehat{A}^{n_0}\widehat{S}_{n_0}(\widehat{A}^*)^{n_0} + (\widehat{A}^{n_0})^2\widehat{S}_{n_0}[(\widehat{A}^*)^{n_0}]^2 + \cdots + (\widehat{A}^{n_0})^{m_0-1}\widehat{S}_{n_0}[(\widehat{A}^*)^{n_0}]^{m_0-1}}_{\text{by } m_0 - 1 \text{ iterations}} \quad \begin{matrix} := \widehat{S}_{(m_0-1)n_0} \\ \end{matrix} \\
 420 \quad &+ (\widehat{A}^{n_0})^{m_0} \cdot \widehat{A}^{q_0+1} \cdot S_l \cdot (\widehat{A}^{q_0+1})^* \cdot [(\widehat{A}^{n_0})^{m_0}]^*.
 \end{aligned}$$

421

422 In the following remark, we will explain how to choose appropriate positive integers m_0, n_0 , and
 423 the natural number l_0 . It is worth noting that the operation of multiplying two square matrices
 424 of size $m \times m$ necessitates a computational effort of m^3 floating-point operations (considering only
 425 matrix multiplication operations).

426 *Remark 4.11.* To minimize the number of floating point values of P_n using the above strategy
 427 of piecewise computation of the matrix series, we should first be clear that the above strategy for
 428 computing P_n requires $3(m_0 + n_0 + \frac{4}{3}q_0)m^3$ flops (considering only matrix multiplication operations,
 429 we analyze the flops under this convention throughout the remainder of the paper). When n is
 430 not a prime number, let us consider the case of $q_0 = 0$, since the positive number n has the
 431 decomposition form $n = m_0n_0 + q_0$, hence $n = m_0n_0$, we can get $m_0 = \frac{n}{n_0}$, hence, we can get
 432 $m_0 + n_0 = \frac{n}{n_0} + n_0 \geq 2\sqrt{n}$, i.e., the minimum value of $m_0 + \frac{3}{2}n_0 + l_0$ is $2\sqrt{n}$, and the condition for
 433 equality to hold is $n_0 = \sqrt{n}$, then we can get $m_0 = \frac{n}{n_0} = \sqrt{n}$. Based on the above analysis, due to
 434 $m_0 = \sqrt{n}$ and $n_0 = \sqrt{n}$ may not be integers, then we can take positive integers m'_0 to replace m_0 and
 435 n'_0 to replace n_0 , as long as m_0n_0 are greater than n (this is because it can also ensure the error
 436 accuracy of P_n we require). Hence, we choose $m'_0 = \lfloor \sqrt{n} \rfloor + 1$ and $n'_0 = \lfloor \sqrt{n} \rfloor + 1$.

437 It should be noted that the number n we calculate may often be a prime number, that is, l_0 is
 438 usually not equal to 0 in some cases. However, the above method of selecting m_0, n_0 , and l_0 also
 439 applies to this situation.

440 Due to

$$\begin{aligned}
 441 \quad m'_0 n'_0 &= (\lfloor \sqrt{n} \rfloor + 1) \cdot (\lfloor \sqrt{n} \rfloor + 1) \\
 442 \quad &= \lfloor \sqrt{n} \rfloor \cdot \lfloor \sqrt{n} \rfloor + \lfloor \sqrt{n} \rfloor + \lfloor \sqrt{n} \rfloor + 1 \\
 443 \quad &\geq (\sqrt{n} - 1) \cdot (\sqrt{n} - 1) + \lfloor \sqrt{n} \rfloor + \lfloor \sqrt{n} \rfloor + 1 \\
 444 \quad &= n - \sqrt{n} - \sqrt{n} + 1 + \lfloor \sqrt{n} \rfloor + \lfloor \sqrt{n} \rfloor + 1 \\
 445 \quad &\geq n,
 \end{aligned}$$

446 this ensures that P_n will satisfy the error precision we require because $m'_0 n'_0 \geq n$.

Therefore, in order to facilitate the operation of the following algorithm and avoid selecting m_0 , n_0 , and l_0 in the following algorithm, we will choose $m'_0 = \lfloor \sqrt{n} \rfloor + 1$ to replace m_0 , $n'_0 = \lfloor \sqrt{n} \rfloor + 1$ to replace n_0 , and $q_0 = 0$.

Building on Algorithm 1, we incorporate this method to propose an improved algorithm that achieves faster computation of $\hat{P}_l^{(n+1)}$ while maintaining accuracy.

Algorithm 2

Input: The initial and final times t_0, t_f , the stable matrix $A \in \mathbb{C}^{m \times m}$, the matrix $B \in \mathbb{C}^{m \times p}$, the matrices $R \in \mathbb{H}_p^+$, $P_f \in \mathbb{H}_m^{++}$, I_m , the number N of nodes, the step time $\delta_T = (t_f - t_0)/N$, and the proper error precision ϵ .

1: Select $q = q_{opt} = \sqrt{\max_{j=1,2,\dots,m} |\lambda_j(A)| \cdot \min_{j=1,2,\dots,m} |\lambda_j(A)|}$.

2: Compute $\hat{A} = (qI_m + A)(qI_m - A)^{-1}$, $\hat{Q} = 2q(qI_m - A)^{-1}BR^{-1}B^*(qI_m - A)^{-*}$, $\rho(\hat{A})$, $S_l = \sum_{k=1}^l \hat{A}^{k-1} \hat{Q} (\hat{A}^*)^{k-1}$, $\xi = \frac{1-\rho(\hat{A})}{2}$, $\beta = (\rho(\hat{A}) + \xi)^2$.

3: If A is Hermitian, set $n = \left\lfloor \frac{\ln\left(\frac{\epsilon(1-\beta)}{\|\hat{Q}\|_2}\right)}{\ln \beta} \right\rfloor$, if not, determine whether $\|(\hat{A})^n\| < [\rho(\hat{A}) + \xi]^n$ holds

when $n = \left\lfloor \frac{\ln\left(\frac{\epsilon(1-\beta)}{\|\hat{Q}\|_2}\right)}{\ln \beta} \right\rfloor$, otherwise, increase the number n until the above inequality holds.

4: According to n , set $m'_0 = \lfloor \sqrt{n} \rfloor + 1$, $n'_0 = \lfloor \sqrt{n} \rfloor + 1$ and $q_0 = 0$, then compute and store $\hat{S}_{n'_0}$, $(\hat{A})^{n'_0}$, $[(\hat{A})^{n'_0}]^*$.

5: Compute $\hat{P}_l^{(n+1)} = \hat{Q} + \hat{S}_{n'_0} + \hat{S}_{(m'_0-1)n'_0} + (\hat{A}^{n'_0})^{m'_0} \cdot \hat{A}^{q_0+1} \cdot S_l \cdot (\hat{A}^{q_0+1})^* \cdot [(\hat{A}^{n'_0})^{m'_0}]^*$.

6: **for** $j = 1, \dots, N$ **do**

7: Compute: $t_j = (j - 1) \cdot \delta_T$;

8: Compute: $P_a(t_j) = \left[e^{-A(t_f-t_j)}(P_f^{-1} + \hat{P}_l^{(n+1)})e^{-A^*(t_f-t_j)} - \hat{P}_l^{(n+1)} \right]^{-1}$.

9: **end for**

Output: $P_a(t_j)$, $1 \leq j \leq N$.

Remark 4.12. This approach for computing the matrix series to derive P_* outperforms traditional CALE solvers, such as the Bartels-Stewart algorithm, in specific scenarios. When computational resources are limited and the coefficient matrix is sparse, each term of $(\hat{A}^*)^k \hat{Q} \hat{A}^k$ can be efficiently calculated using sparse matrix multiplication, storing only non-zero elements and allowing early truncation based on desired accuracy. In contrast, methods like Bartels-Stewart rely on dense matrix decompositions, such as Schur decomposition, which fail to capitalize on matrix sparsity, incurring $O(n^3)$ computational complexity and $O(n^2)$ memory usage. These methods require completing all computations in a single step, offering no intermediate approximations and limiting their suitability for real-time applications. Conversely, with sufficient resources, the proposed algorithm supports parallelization, further enhancing computational efficiency.

Remark 4.13. The theoretical minimum precision varies across floating-point formats; for instance, single-precision floating-point numbers have a minimum precision of approximately 5.96e-8, while double-precision numbers achieve about 1.11e-16. Consequently, when implementing the proposed algorithm, the choice of ϵ must account for the floating-point precision used, as it directly affects the value of n and, thus, the algorithm's efficiency. For example, when using single-precision floating-point numbers for Algorithm 1 and Algorithm 2, ϵ should be set larger than 5.96e-8. Set-

ting ϵ below this threshold will not improve the accuracy of n or P_n beyond the single-precision limit of 5.96e-8, resulting in unnecessary computational overhead. Therefore, in practical applications, ϵ should be chosen to exceed the theoretical minimum precision corresponding to the selected floating-point format.

We now analyze the computational complexity of Algorithm 1 and Algorithm 2 when $S_l = S_1 = \hat{Q}$ as presented in this paper. Let us recall that the multiplication of two $m \times m$ square matrices requires m^3 floating-point operations.

Remark 4.14. Let $Flop_{Alg.1}$ and $Flop_{Alg.2}$ denote the minimum number of flops required by Algorithm 1 and Algorithm 2, respectively. For Algorithm 1, we have

$$Flop_{Alg.1} = \left(19 + 2n + \frac{13}{3}N\right) m^3.$$

For Algorithm 2, we obtain

$$Flop_{Alg.2} = \left(17 + 6\sqrt{n} + \frac{13}{3}N\right) m^3.$$

Thus, for sufficiently large n , Algorithm 2 may require fewer flops than Algorithm 1, enhancing computational efficiency.

Remark 4.15. As is well known, solving the CALE using the Bartels–Stewart algorithm requires approximately $\frac{35}{2}m^3$ floating-point operations. In contrast, the computational costs of our proposed algorithms for solving the same equation are $\left(\frac{23}{3} + 2n\right)m^3$ and $\left(\frac{17}{3} + 6\sqrt{n}\right)m^3$, respectively. When the parameter n is relatively small, our methods are significantly more efficient than the Bartels–Stewart algorithm in terms of computational complexity.

5. Numerical Experiments. In this section, we present numerical experiments to illustrate the effectiveness and superiority of our derived results. All experiments were conducted using MATLAB 2019b on an Intel Xeon 8358 processor with a 2.6GHz speed and 512 GB of memory. The algorithms were implemented using the following MATLAB functions:

- expm:** Computes the exponential of a square matrix using a scaling and squaring algorithm with a Padé approximation.
- lyap:** Solves the continuous algebraic Lyapunov equation $AP + PA^* + Q = 0$, where the command `lyap(A, Q)` returns the solution matrix P .
- eig:** Numerically calculates the all eigenvalues of a matrix.

To evaluate the performance of different methods for solving the RDE given in Equation (1.1), we adopted the following comparison criteria:

1. **CPU Time:** The total duration of time that the central processing unit spends executing a specific algorithm or program, measured in seconds. It represents the computational effort required by the CPU to complete the task, excluding time spent on other system activities such as input/output operations or waiting for external resources.
2. **Maximum Relative Error Norm (REN):** We define the maximum relative error norm over the time steps $j = 1, 2, \dots, N$ as:

$$REN = \max_{j=1,2,\dots,N} \frac{\|P_a(t_j) - P(t_j)_{Ref}\|_2}{\|P(t_j)_{Ref}\|_2}.$$

3. Defects in Hermitian and Positive Semidefiniteness: We assess the Hermitian and positive semidefiniteness of the approximate solution at the final time step t_N using:

$$d_{Her} = \frac{\|P_a(t_N) - P_a^*(t_N)\|_F}{\|P(t_N)_{Ref}\|_F}, \quad d_{psd} = \frac{\|P_a(t_N) - \widetilde{P}_a(t_N)\|_F}{\|P(t_N)_{Ref}\|_F},$$

where $P_a^*(t_N)$ is the conjugate transpose of $P_a(t_N)$, and $\widetilde{P}_a(t_N)$ is the nearest Hermitian positive semidefinite matrix to $P_a(t_N)$, computed using the algorithm from [20].

Experiment 5.1 In this experiment, we primarily compare the CPU time, maximum relative error norm, and the defects in Hermitian and positive semidefiniteness of Algorithm 1, Algorithm 2, the BDF method, the ROS method, and several MATLAB solvers (`ode45`, `ode15s`, and `ode23tb`) for solving the RDE (1.1). The test matrices A with dimension m considered in this experiment are sourced from MATLAB's built-in test matrices or the Matrix Market, and they are Hermitian. Additionally, the matrices are defined as $P_f = I_m$, $B = I_m$ and $R = I_m$, $S_1 = 2q(qI_m - A)^{-1}BR^{-1}B^*(qI_m - A)^{-*}$. For Algorithm 1 and Algorithm 2, the error precision ϵ is set to 1×10^{-12} . The following parameters are used with the `odeset` function: `RelTol` = 1×10^{-11} and `AbsTol` = 1×10^{-12} . In this test, the reference solution $P(t)_{Ref}$ is computed using the Anderson-Moore method as described in [2].

In the following experiment, we consider the time interval $[0, t_f]$ with $t_f = 1$, initially discretized with $N = 50$ nodes, corresponding to a step size $\delta_T = 0.02$. The CPU time, relative error norm, and defects in Hermitian and positive semidefiniteness of the solutions obtained by these methods are presented in Table 1, Fig. 1, and Fig. 2.

Table 1: The results for different methods when $t_f = 1$ and $\delta_T = 0.02$

Test Problem	Method	CPU time (seconds)	REN	d_{Her}	d_{psd}
$A = -mesh1em6$ ($m = 48$)	Algorithm 1 ($q = 0.1$)	0.5384	4.5677e-12	3.7517e-17	7.8522e-16
	Algorithm 1 ($q = q_{opt} = 2.9156$)	0.0274	1.6745e-12	2.6320e-17	7.9162e-16
	Algorithm 1 ($q = 1000$)	0.5637	1.3787e-10	8.4923e-17	8.6105e-16
	Algorithm 2 ($q = 0.1$)	0.0406	1.8104e-12	2.6758e-17	7.5147e-16
	Algorithm 2 ($q = q_{opt} = 2.9156$)	0.0387	1.0263e-12	2.4756e-17	8.0549e-16
	Algorithm 2 ($q = 1000$)	0.0407	1.2004e-12	2.4387e-17	7.9397e-16
	BDF	2.6097	0.1140	0	7.9825e-16
	ROS	0.3100	0.0325	0	8.1994e-16
	ode45	0.8104	2.0790e-12	0	7.8605e-16
	ode15s	22.2634	1.6944e-10	1.7765e-19	8.8575e-16
	ode23tb	511.8134	8.6825e-09	1.7482e-24	8.2541e-16
	Algorithm 1 ($q = 0.1$)	1.0263	7.7633e-11	2.0584e-16	8.2854e-16
	Algorithm 1 ($q = q_{opt} = 0.9573$)	0.1334	5.0863e-11	1.6076e-16	8.3207e-16
	Algorithm 1 ($q = 10$)	0.8934	4.8337e-11	1.6212e-16	8.4764e-16
$A = -gallery('poisson', 12)$ ($m = 144$)	Algorithm 2 ($q = 0.1$)	0.0948	3.0120e-11	2.3902e-16	8.3620e-16
	Algorithm 2 ($q = q_{opt} = 0.9573$)	0.1127	2.7262e-11	1.6023e-16	8.4113e-16
	Algorithm 2 ($q = 10$)	0.1167	3.5622e-11	1.5817e-16	8.6094e-16
	BDF	12.3276	0.1108	0	9.2120e-16
	ROS	0.3178	0.0310	0	8.2855e-16
	ode45	0.7824	7.6636e-11	0	8.6896e-16
	ode15s	6325.7613	1.4899e-10	5.2930e-19	8.3573e-16
	ode23tb	Fail	Fail	Fail	Fail
	Algorithm 1 ($q = 0.05$)	2581.2803	3.1711e-11	0	0
	Algorithm 1 ($q = q_{opt} = 0.0918$)	2485.5779	1.8339e-13	0	0
	Algorithm 1 ($q = 0.15$)	4337.6269	1.8339e-13	0	0
	Algorithm 2 ($q = 0.05$)	69.9938	6.4398e-16	0	0
	Algorithm 2 ($q = q_{opt} = 0.0918$)	54.6495	1.8339e-13	0	0
	Algorithm 2 ($q = 0.15$)	64.5017	1.8339e-13	0	0
$A = -bcsstm11$ ($m = 1473$)	BDF	Fail	Fail	Fail	Fail
	ROS	Fail	Fail	Fail	Fail
	ode45	Fail	Fail	Fail	Fail
	ode15s	Fail	Fail	Fail	Fail
	ode23tb	Fail	Fail	Fail	Fail
	ode23tb	Fail	Fail	Fail	Fail

It is evident that Algorithm 1 and Algorithm 2 consistently outperform other methods in terms of CPU time when the parameter $q = q_{opt}$ (the average runtime performance in second, with

statistics calculated over 30-times runs of these algorithms). This suggests that selecting the optimal q accelerates the computation of the numerical solution to the RDE (1.1). The relative error norms for Algorithm 1 and Algorithm 2 consistently remain below 10^{-9} , surpassing other methods in most scenarios. This highlights the effectiveness and superiority of both algorithms. Additionally, Algorithm 1 and Algorithm 2 exhibit exceptional performance in terms of Hermitian and positive semidefiniteness defects. Furthermore, as matrix size increases in these experiments, other methods often encounter memory limitations or excessive CPU time beyond acceptable thresholds, whereas Algorithm 1 and Algorithm 2 remain effective, further underscoring their advantages.

To visually illustrate the accuracy of various methods, we present the relative error norms for the test matrices $A = -\text{meshlem6}$ and $A = -\text{gallery}(\text{'poisson'}, 12)$ in Figure 1.

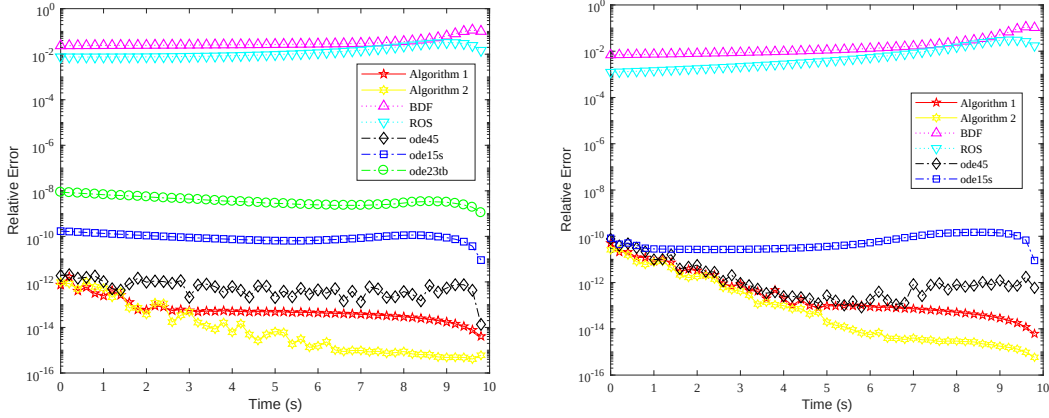


Fig. 1: **Left:** the relative error norm for $A = -\text{meshlem6}$ when $t_f = 1$ with $\delta_T = 0.02$; **right:** the relative error norm for $A = -\text{gallery}(\text{'poisson'}, 12)$ when $t_f = 1$ with $\delta_T = 0.02$.

From Figure 1, it is observed that the relative error norms for Algorithm 1 and Algorithm 2 consistently maintain a stable level below 10^{-9} in this case. In contrast, other methods exhibit poor performance in relative error norms at certain time nodes. This demonstrates the superior relative error precision of our proposed algorithms.

Experiment 5.2 The primary distinction from Experiment 5.1 lies in the test matrices A of dimension m , which are non-Hermitian and stable in this case.

Suppose that

$$K = \begin{bmatrix} -2 & 8 & -3 \\ -4 & -2 & 9 \\ 3 & -2 & -5 \end{bmatrix},$$

is fixed, we select $K_1 = \text{gallery}(\text{'lehmer'}, k_1)$, $K_2 = \text{gallery}(\text{'leslie'}, k_2)$, $K_3 = \text{gallery}(\text{'prolate'}, k_3)$ in this experiment, then let us define the coefficient matrices A_i as follows

$$A_i = \alpha \cdot I_m + \beta \cdot K_i \otimes K$$

where $m = 3k_i$, and the real number $\alpha < 0$ and $\beta > 0$. The time interval $[0, t_f]$ with $t_f = 1$, initially discretized with $N = 50$ nodes, corresponding to a step size $\delta_T = 0.02$. The CPU time,

relative error norm, and defects in Hermitian and positive semidefiniteness of the solutions obtained by these methods are presented in Table 2 and Figure 2.

Table 2: The results for different methods when $t_f = 1$ and $\delta_T = 0.02$

Test Problem	Method	CPU time (seconds)	REN	d_{Her}	d_{psd}
$A = -3 \cdot I_{66} + 0.1 \cdot K_1 \otimes K$ ($m = 66$)	Algorithm 1 ($q = 0.1$)	1.0567	3.8995e-12	8.3228e-17	8.8303e-16
	Algorithm 1 ($q = q_{opt} = 6.1467$)	0.7000	2.6338e-14	4.7451e-17	9.3180e-16
	Algorithm 1 ($q = 1000$)	3.4935	2.6955e-11	3.5455e-16	9.3625e-16
	Algorithm 2 ($q = 0.1$)	0.5387	2.2344e-14	5.2168e-17	8.9332e-16
	Algorithm 2 ($q = q_{opt} = 6.1467$)	0.0895	3.2019e-14	4.7467e-17	9.2385e-16
	Algorithm 2 ($q = 1000$)	0.1026	4.8959e-14	5.4026e-17	8.7254e-16
	BDF	9.0730	0.1577	0	9.4020e-16
	ROS	1.0128	0.1337	0	8.8553e-16
	ode45	0.6153	5.4783e-11	0	1.0059e-15
	ode15s	2799.6328	5.6764e-09	2.9557e-19	9.2435e-16
	ode23tb	Fail	Fail	Fail	Fail
	ode23tb	Fail	Fail	Fail	Fail
$A = -3 \cdot I_{126} + 0.1 \cdot K_1 \otimes K$ ($m = 126$)	Algorithm 1 ($q = 0.1$)	6.4344	1.1075e-15	8.0984e-18	5.0917e-16
	Algorithm 1 ($q = q_{opt} = 2.9764$)	0.2681	2.9281e-14	4.8977e-17	9.4342e-16
	Algorithm 1 ($q = 1000$)	9.7790	1.1038e-15	6.0441e-18	5.4844e-16
	Algorithm 2 ($q = 0.1$)	10.4429	1.1081e-15	8.3451e-18	5.0773e-16
	Algorithm 2 ($q = q_{opt} = 2.9764$)	0.2247	3.3738e-14	4.9309e-17	8.9092e-16
	Algorithm 2 ($q = 1000$)	7.3069	1.1039e-15	5.9742e-18	5.0837e-16
	BDF	1.9123	0.1044	0	9.4071e-16
	ROS	1.6007	0.0420	1.7281e-16	9.1181e-16
	ode45	0.6122	7.2205e-12	6.7478e-19	1.0424e-15
	ode15s	914.4613	4.6971e-10	1.2213e-18	9.2687e-16
	ode23tb	Fail	Fail	Fail	Fail
	ode23tb	Fail	Fail	Fail	Fail
$A = -3 \cdot I_{1080} + 0.1 \cdot K_3 \otimes K$ ($m = 1080$)	Algorithm 1 ($q = 0.1$)	97.9520	4.9981e-12	2.4473e-17	9.2094e-16
	Algorithm 1 ($q = q_{opt} = 3.2453$)	11.9898	3.6328e-15	2.3666e-17	9.3020e-16
	Algorithm 1 ($q = 1000$)	723.7802	6.2590e-11	9.7801e-17	9.5707e-16
	Algorithm 2 ($q = 0.1$)	14.7862	3.7980e-15	2.4229e-17	9.3006e-16
	Algorithm 2 ($q = q_{opt} = 3.2453$)	11.5125	3.6382e-15	2.3542e-17	9.4244e-16
	Algorithm 2 ($q = 1000$)	23.5922	2.2024e-14	2.3844e-17	9.4629e-16
	BDF	Fail	Fail	Fail	Fail
	ROS	Fail	Fail	Fail	Fail
	ode45	Fail	Fail	Fail	Fail
	ode15s	Fail	Fail	Fail	Fail
	ode23tb	Fail	Fail	Fail	Fail
	ode23tb	Fail	Fail	Fail	Fail

From the above table, it demonstrates that Algorithms 1 and 2 consistently outperform other methods in computational efficiency when using the optimal parameter $q = q_{opt}$ (average runtime in seconds over 30 trials). This confirms that optimal q selection significantly accelerates solving the RDE (1.1) numerically. The relative error norms of the numerical solutions obtained by Algorithm 1 and Algorithm 2 remain below 10^{-10} , demonstrating the effectiveness of these algorithms. Similarly, as the dimension of the test matrices increases, the memory usage and CPU time of other methods exceed acceptable limits. This highlights the superiority of Algorithm 1 and Algorithm 2 over certain established classical methods. Additionally, we observe that when the test matrices are non-Hermitian, other methods exhibit significantly poor performance in terms of relative error norms. In contrast, the numerical solutions derived from Algorithm 1 and Algorithm 2 maintain excellent performance for the defects in Hermitian and positive semidefiniteness.

Subsequently, we present the following figures, which illustrate the relative error norms of the numerical solutions obtained by various methods for $A = -3 \cdot I_{66} + 0.1 \cdot K_1 \otimes K$ with dimension $m = 66$ and $A = -3 \cdot I_{126} + 0.1 \cdot K_1 \otimes K$ with dimension $m = 126$, evaluated over $t_f = 1$ with a step size $\delta_T = 0.02$.

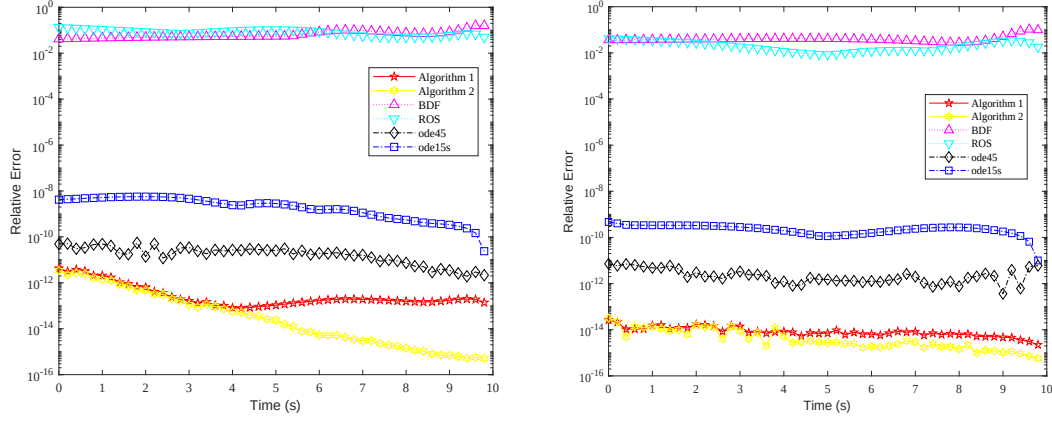


Fig. 2: **Left:** the relative error norm for $A = -3 \cdot I_{66} + 0.1 \cdot K_1 \otimes K$ with dimension $m = 66$ when $t_f = 1$ with $\delta_T = 0.02$; **right:** the relative error norm for $A = -3 \cdot I_{126} + 0.1 \cdot K_1 \otimes K$ with dimension $m = 126$ when $t_f = 1$ with $\delta_T = 0.02$.

From Figure 2, it is observed that the relative error norms for Algorithm 1 and Algorithm 2 consistently maintain a stable level below 10^{-14} in this case. In contrast, other methods exhibit poor performance in relative error norms at certain time nodes. This underscores the advantages of our proposed algorithms.

6. Conclusion. In this paper, we have proposed iterative algorithms to solve the Riccati matrix differential equation. A comprehensive analysis of the convergence, error, and computational complexity of these algorithms has been presented. The effectiveness and superiority of the proposed algorithms have been validated through a series of numerical experiments. Future work will concentrate on developing more efficient algorithms for solving matrix differential equations.

Disclosure statement. The authors declare no conflicts of interest.

Data Availability Statement. The data used to support the findings of this study are included within this article.

Acknowledgments. This work was supported in part by the High Performance Computing Platform of Xiangtan University.

REFERENCES

- [1] H. ABOU-KANDIL, G. FREILING, V. IONESCU, AND G. JANK, *Matrix Riccati equations in control and systems theory*, Birkhäuser, 2012.
- [2] B. D. ANDERSON AND J. B. MOORE, *Linear optimal control prentice hall*, Englewood Cliffs, (1971).
- [3] A. C. ANTOUNAS, *Approximation of large-scale dynamical systems*, SIAM, 2005.
- [4] M. BEHR, P. BENNER, AND J. HEILAND, *Galerkin trial spaces and davison-maki methods for the numerical solution of differential riccati equations*, Applied Mathematics and Computation, 410 (2021), p. 126401.
- [5] P. BENNER AND R. BYERS, *An exact line search method for solving generalized continuous-time algebraic riccati equations*, IEEE Transactions on Automatic Control, 43 (1998), pp. 101–107.
- [6] P. BENNER AND N. LANG, *Peer methods for the solution of large-scale differential matrix equations*, arXiv preprint arXiv:1807.08524, (2018).

- [7] P. BENNER AND H. MENA, *Bdf methods for large-scale differential riccati equations*, Proc. of Mathematical Theory of Network and Systems, MTNS, 2004 (2004), p. 10.
- [8] P. BENNER AND H. MENA, *Rosenbrock methods for solving riccati differential equations*, IEEE Transactions on Automatic Control, 58 (2013), pp. 2950–2956.
- [9] P. BENNER AND H. MENA, *Numerical solution of the infinite-dimensional lqr problem and the associated riccati differential equations*, Journal of Numerical Mathematics, 26 (2018), pp. 1–20.
- [10] H. CHEN AND A. BORZI, *Positivity preserving exponential integrators for differential riccati equations*, Journal of Scientific Computing, 96 (2023), p. 50.
- [11] H. CHEN AND A. BORZI, *Low-rank exponential integrators for stiff differential riccati equations*, Advances in Computational Mathematics, 51 (2025), p. 17.
- [12] H. CHEN AND Y. WANG, *Modified douglas splitting method for differential matrix equations*, Journal of Computational and Applied Mathematics, 384 (2021), p. 113162.
- [13] E.-W. CHU, H.-Y. FAN, AND W.-W. LIN, *A structure-preserving doubling algorithm for continuous-time algebraic riccati equations*, Linear algebra and its applications, 396 (2005), pp. 55–80.
- [14] H. DAI AND Z.-Z. BAI, *On eigenvalue bounds and iteration methods for discrete algebraic riccati equations*, Journal of Computational Mathematics, (2011), pp. 341–366.
- [15] L. DIECI, *Numerical integration of the differential riccati equation and some related issues*, SIAM Journal on numerical analysis, 29 (1992), pp. 781–815.
- [16] F. A. FARUQI, P. BELOBABA, J. COOPER, AND A. SEABRIDGE, *Differential game theory with applications to missiles and autonomous systems guidance*, John Wiley & Sons, 2017.
- [17] T. GU, H. AN, X. LIU, AND X. XU, *Iterative methods and preprocessing techniques*, 2015.
- [18] E. HANSEN AND T. STILLFJORD, *Convergence analysis for splitting of the abstract differential riccati equation*, SIAM Journal on Numerical Analysis, 52 (2014), pp. 3128–3139.
- [19] M. HEYOUNI AND K. JBILOU, *An extended block arnoldi algorithm for large-scale solutions of the continuous-time algebraic riccati equation*, Electronic Transactions on Numerical Analysis, 33 (2009), pp. 53–62.
- [20] N. J. HIGHAM, *Functions of matrices: theory and computation*, SIAM, 2008.
- [21] R. A. HORN AND C. R. JOHNSON, *Topics in matrix analysis*, Cambridge university press, 1994.
- [22] R. A. HORN AND C. R. JOHNSON, *Matrix analysis*, Cambridge university press, 2012.
- [23] A. ICHIKAWA AND H. KATAYAMA, *Remarks on the time-varying H_∞ riccati equations*, Systems & control letters, 37 (1999), pp. 335–345.
- [24] O. L. R. JACOBS, *Introduction to control theory / O.L.R. Jacobs.*, Oxford University Press, Oxford ;, 2nd ed. ed., 1993.
- [25] G. KIRSTEN AND V. SIMONCINI, *Order reduction methods for solving large-scale differential matrix riccati equations*, SIAM Journal on Scientific Computing, 42 (2020), pp. A2182–A2205.
- [26] A. LAUB, *A schur method for solving algebraic riccati equations*, IEEE Transactions on automatic control, 24 (2003), pp. 913–921.
- [27] J. LIU AND J. ZHANG, *The open question of the relation between square matrix's eigenvalues and its similarity matrix's singular values in linear discrete system*, International Journal of Control, Automation and Systems, 9 (2011), pp. 1235–1241.
- [28] J. LIU, J. ZHANG, AND F. LUO, *Newton's method for the positive solution of the coupled algebraic riccati equation applied to automatic control*, Computational and Applied Mathematics, 39 (2020), p. 113.
- [29] A. MÅLQVIST, A. PERSSON, AND T. STILLFJORD, *Multiscale differential riccati equations for linear quadratic regulator problems*, SIAM Journal on Scientific Computing, 40 (2018), pp. A2406–A2426.
- [30] H. MENA, *Numerical solution of differential riccati equations arising in optimal control problems for parabolic partial differential equations*, Ph. D. thesis, Escuela Politecnica Nacional, (2007).
- [31] H. MENA, A. OSTERMANN, L.-M. PFURTSCHELLER, AND C. PIAZZOLA, *Numerical low-rank approximation of matrix differential equations*, Journal of Computational and Applied Mathematics, 340 (2018), pp. 602–614.
- [32] A. OSTERMANN, C. PIAZZOLA, AND H. WALACH, *Convergence of a low-rank lie-trotter splitting for stiff matrix differential equations*, SIAM Journal on Numerical Analysis, 57 (2019), pp. 1947–1966.
- [33] T. STILLFJORD, *Low-rank second-order splitting of large-scale differential riccati equations*, IEEE Transactions on Automatic Control, 60 (2015), pp. 2791–2796.
- [34] T. STILLFJORD, *Adaptive high-order splitting schemes for large-scale differential riccati equations*, Numerical Algorithms, 78 (2018), pp. 1129–1151.
- [35] L. TIAN, *Research on control and solving method of pursuit-evasion game for spacecraft*, Harbin Institute of Technology, (2021).
- [36] I. P. VALERY, A. U. ANDREY, AND V. SAVKIN, *Robust control design using H_∞ methods*, Springer Science & Business Media, (2000).
- [37] J. ZHANG AND S. LI, *The structure-preserving doubling numerical algorithm of the continuous coupled algebraic*

- 644 *riccati equation*, International Journal of Control, Automation and Systems, 18 (2020), pp. 1641–1650.
645 [38] J. ZHANG, J. LIU, AND F. LUO, *A class of fixed point iteration for the coupled algebraic riccati equation*, Journal
646 of Applied Mathematics and Computing, 68 (2022), pp. 4119–4133.